HackNEX — Frontend Design & UI Specification Document

Project: HackNEX
Organizer: NEXUS Club, Karunya Institute of Technology and Sciences
Event: HackNEX 2026
Event Dates: October 7–9, 2026
Mode: Offline
Expected Participants: 1,500+
Registration Fee: ₹600/team
Frontend: React-based
Design Reference: 21st.dev components, including Neural Noise
Primary Design: Futuristic, Premium, Minimal, Dark, Glassmorphic, Experimental

---

1. Document Purpose

This document defines the complete frontend architecture, user interface, visual design system, page structure, interactions, responsive behavior, component strategy, and frontend implementation requirements for the HackNEX website.

The document serves as the primary frontend blueprint for development.

The frontend must provide two distinct experiences:

1. **Public HackNEX website**
2. **Authenticated participant/registration interface and administration interface**

The public website focuses on presenting HackNEX and converting visitors into registered teams.

---

2. Design Philosophy

HackNEX should not look like a conventional college event website.

The visual identity should communicate:

- Technology
- Innovation

- Competition
- Engineering
- Futurism
- Premium quality
- NEXUS Club identity

The design should use **21st.dev components as inspiration**, but the final website must have a unique HackNEX visual identity.

The website should avoid excessive effects that negatively impact performance.

Design keywords

Futuristic · Premium · Dark · Minimal · Experimental · Technical · Immersive

---

3. Reference Design System

21st.dev will be used as the primary component inspiration source.

Reference:

21st.dev Neural Noise Reference

Components can be adapted for:

- Background effects
- Noise effects
- Cards
- Buttons
- Text animations
- Navigation
- Section transitions
- Interactive elements

Important rule

The website must **not become a collection of unrelated 21st.dev components**.

All components must follow one common:

- Typography system
- Spacing system
- Border system
- Radius system
- Animation language
- Color system

- Interaction language

---

4. Website Architecture

The frontend will contain the following major areas:

```
HackNEX
│
├── Loading Screen
│
├── Public Website
│   ├── Home
│   ├── About
│   ├── Domains
│   ├── Details
│   ├── Schedule
│   ├── Prizes
│   ├── Sponsors
│   ├── Venue
│   ├── FAQ
│   └── Footer
│
├── Authentication
│   ├── Login
│   ├── Signup
│   ├── Email Verification
│   ├── Forgot Password
│   └── Reset Password
│
├── Registration
│   ├── Team Details
│   ├── Captain Details
│   ├── Member 2
│   ├── Member 3
│   ├── Member 4
│   ├── Review
│   └── Payment
│
└── Admin
    ├── Dashboard
    ├── Teams
    ├── Participants
    ├── Payments
    ├── Domains
```

```
├── Schedule
├── Prizes
├── Sponsors
├── Gallery
├── FAQ
└── Announcements
```

---

5. Frontend Routes

Public Routes

/

Landing page.

/login
/signup
/verify-email
/forgot-password
/reset-password

Authentication.

/register
/register/team
/register/captain
/register/member-2
/register/member-3
/register/member-4
/register/review
/register/payment
/register/success

Registration workflow.

Admin Routes

/admin
/admin/teams
/admin/participants
/admin/payments
/admin/domains
/admin/schedule
/admin/prizes
/admin/sponsors
/admin/gallery

/admin/faq
/admin/announcements

Admin routes must be protected.

---

6. Loading Screen

The first visual shown when visiting HackNEX will be the existing:

loadingscreen.mp4

The video will be used as the official HackNEX loading-screen animation.

Loading flow

Website Request
↓
Loading Screen
↓
Essential Assets Loaded
↓
Transition Animation
↓
HackNEX Landing Page

Requirements

- Video should not block the website indefinitely.
- Loading should have a fallback.
- Video should be optimized for web delivery.
- Mobile behavior should be considered.
- The loading screen should transition smoothly into the hero.
- The loading screen should appear only when necessary rather than forcing long waits on every navigation.

---

7. Navigation Bar

The desktop navbar will contain:

HACKNEX

Home

About
Domains
Schedule
Prizes
Sponsors
FAQ

[REGISTER NOW]

Sections such as Details and Venue will remain accessible through the page scroll and relevant CTAs rather than overcrowding the navbar.

Navbar behavior

Initial state:

Transparent / floating over hero

After scrolling:

Glassmorphic / blurred background

Navbar requirements

- Sticky navigation
- Smooth scrolling
- Active section indication
- Responsive mobile navigation
- Register Now CTA
- Logo/brand identity
- Accessible keyboard navigation

---

8. Mobile Navigation

On mobile:

HACKNEX ☰

Clicking the menu opens a navigation drawer.

Home
About
Domains
Schedule
Prizes
Sponsors

FAQ

REGISTER NOW

The drawer should animate smoothly.

---

9. Hero / Home Section

The Hero is the primary conversion section.

Recommended structure:

NEXUS CLUB PRESENTS

HACKNEX

[TAGLINE]

OCTOBER 7 — 9, 2026

Karunya Institute of Technology and Sciences

[REGISTER NOW]   [EXPLORE HACKNEX]

Visual direction

The hero should combine:

- Large typography
- Animated background
- Neural/noise visual language
- Grid elements
- Subtle particles
- Glassmorphic elements
- HackNEX branding

The hero must immediately communicate:

What: HackNEX
Who: NEXUS Club
When: October 7–9, 2026
Where: KITS
Action: Register

---

10. Hero Animation

Animation intensity:

Level 2–3

The hero can use:

- Neural noise
- Particle effects
- Subtle parallax
- Text reveal
- Glow
- Grid animation
- Mouse interaction
- Hover effects

However:

No unnecessary continuous heavy animations.

Performance takes priority.

---

11. Tagline

The official tagline is currently undecided.

The frontend must therefore support a configurable tagline.

Potential future tagline:

Build. Innovate. Transform.

The final tagline will be provided later.

---

12. Countdown

A countdown component is recommended.

Example:

HACKNEX BEGINS IN

12
DAYS

08
HOURS

42
MINUTES

19
SECONDS

The countdown should automatically use the configured event date.

After the event begins, the component should change state rather than displaying a negative countdown.

---

13. About Section

The About section should contain:

About HackNEX

Brief introduction to the event.

Agenda

Explain the purpose and overall flow.

Why Participate?

Highlight:

- Problem solving
- Innovation
- Networking
- Collaboration
- Competition
- Recognition
- Prizes

---

14. HackNEX Statistics

A statistics section should display key numbers.

Initial structure:

1500+
Participants

375+
Teams

₹1.5L
Prize Pool

3
Days

The prize pool remains configurable because final prize information has not yet been finalized.

Statistics should use animated counters where appropriate.

---

15. Domains Section

The Domains section will initially use mock data.

Potential domains:

AI / ML
Cybersecurity
FinTech
Healthcare
Web3
Sustainability

Final domains will be supplied later.

UI

Recommended:

Interactive futuristic card/bento layout.

Each domain card can contain:

- Icon
- Domain name
- Short description
- Visual accent
- Hover animation

---

16. Details Section

The Details section will use a hybrid approach.

Website presentation

Important information will be displayed directly on the website.

Document

The complete official details document will also be made available through:

[VIEW DETAILS]
[DOWNLOAD DETAILS]

This ensures users don't have to download the document just to understand basic information.

---

17. Schedule Section

Schedule will use a vertical interactive timeline.

Example:

```
OCT 7
 |
 ● Registration
 |
 ● Opening Ceremony
 |
 ● Hacking Begins
 |
```

OCT 8
|
● Development
|
● Mentoring
|
OCT 9
|
● Submission
|
● Judging
|
● Results

Timeline features

- Day grouping
- Time
- Event title
- Description
- Optional location
- Highlighted current/upcoming event
- Mobile-friendly layout

Actual schedule will be configurable later.

---

18. Prizes Section

Prize information will be presented using large premium cards.

Possible structure:

1st Place
₹XX

2nd Place
₹XX

3rd Place
₹XX

Additional categories can be added later.

The component should support:

- Overall prizes
- Runner-up prizes
- Domain prizes
- Special awards

---

19. Sponsors Section

Sponsor logos will be displayed according to sponsor tier.

Potential structure:

TITLE SPONSOR

[LOGO]

GOLD SPONSORS

[LOGO] [LOGO]

SILVER SPONSORS

[LOGO] [LOGO] [LOGO]

Sponsor data must eventually be manageable through the admin panel.

Images will be delivered through **Cloudinary**.

---

20. Venue Section

Venue:

Karunya Institute of Technology and Sciences, Coimbatore

The section will contain:

- Campus image
- Venue image
- Address
- Directions
- Google Maps
- Transportation information

Example:

WHERE HACKNEX HAPPENS

Karunya Institute of Technology
Coimbatore

[Campus Image]

[OPEN IN GOOGLE MAPS]

---

21. FAQ Section

FAQ will use categorized accordion UI.

Registration

- Who can participate?
- What is the team size?
- Can students from different colleges form a team?
- Can students from different departments form a team?

Payment

- What is the registration fee?
- How is payment completed?
- How is payment verified?
- Is the registration fee refundable?

Event

- When is HackNEX?
- Where is HackNEX?
- Is HackNEX online or offline?

Team

- Can one participant join multiple teams?
- Can the team information be changed?
- Can I register without completing my team?

Technical

- What domains are available?
- What should participants bring?

- What technologies can be used?

Communication

- How will important announcements be communicated?

---

22. Final Registration CTA

Before the footer, a large CTA section should appear.

Example:

READY TO BUILD SOMETHING GREAT?

HACKNEX 2026

October 7–9

[REGISTER NOW]

This should be one of the strongest conversion points on the website.

---

23. Footer

Footer structure:

HACKNEX
NEXUS CLUB
Karunya Institute of Technology and Sciences

HackNEX
About
Domains
Schedule
Prizes
FAQ
Register

NEXUS CLUB

LinkedIn
Instagram

KITS
Official Website

The actual official URLs will be added later.

---

24. Authentication UI

Authentication pages:

Login
Signup
Email Verification
Forgot Password
Reset Password

All authentication pages should retain the HackNEX visual identity.

Desktop

Split-screen layout:

```
┌──────────────────────┬──────────────────────┐
│            │            │
│  HACKNEX      │   LOGIN      │
│  VISUAL      │            │
│            │  Email       │
│            │  Password      │
│            │            │
│            │  [ Login ]     │
│            │            │
└──────────────────────┴──────────────────────┘
```

Mobile

Centered form layout.

---

25. Signup

Signup form:

Full Name
Email
Password
Confirm Password

[Create Account]

After signup:

Account Created

Check your email to verify your account.

---

26. Email Verification

The verification page should support:

Pending

CHECK YOUR EMAIL

We've sent a verification link to your email.

[Resend Email]

Success

EMAIL VERIFIED

Your account is ready.

[Continue]

Expired

VERIFICATION LINK EXPIRED

[Send New Verification Email]

---

27. Registration Architecture

Registration is restricted to authenticated users.

Unauthenticated user:

REGISTER NOW
↓
Login / Signup
↓
Email Verification
↓
Registration

Authenticated user:

REGISTER NOW
↓
Registration Wizard

---

28. Registration Wizard

The registration UI will be a multi-step wizard.

Team
↓
Captain
↓
Member 2
↓
Member 3
↓
Member 4
↓
Review
↓
Payment
↓
Success

A progress indicator should be visible.

Example:

● Team —— ● Captain —— ● Members —— ○ Review —— ○ Payment

---

29. Team Details

Fields:

Team Name

Team names must be unique.

---

30. Participant Information

Every participant must provide:

Name
Email
Phone Number
College
Department
Year of Study
LinkedIn URL
Food Preference

Food preference:

○ Vegetarian
○ Non-Vegetarian

The captain will enter the information for all four participants.

---

31. Team Composition

Exactly four participants:

Captain
Member 2
Member 3
Member 4

Teams can contain participants from:

- Different colleges
- Different departments

A participant cannot belong to multiple teams.

---

32. Review Page

Before payment, the user must see a complete summary.

Example:

TEAM
Hack Warriors

CAPTAIN
Name
Email
College
Department
Year

MEMBER 2
...

MEMBER 3
...

MEMBER 4
...

PAYMENT
Registration Fee
₹600

Actions:

[Edit]
[Confirm & Continue]

---

33. Payment UI

Payment is the final registration stage.

Registration Fee

₹600 / Team

[Proceed to Payment]

Payment states:

Pending

Payment Pending

Processing

Verifying Payment...

Successful

Payment Successful

Failed

Payment Failed

[Retry Payment]

Registration success

Registration confirmation must only occur after successful payment verification.

---

34. Registration Success

Successful state:

HACKNEX REGISTRATION COMPLETE

Team: Hack Warriors

Registration ID:
HNX-XXXXXX

Payment: Verified

A confirmation email has been sent to the registered email address.

Further HackNEX information will be communicated through email.

---

35. Participant Website Interaction After Registration

Participants do **not** need a full event dashboard.

The website's primary participant functionality is:

> Registration.

All future event information and announcements will be communicated through registered email.

Therefore, unnecessary participant dashboard functionality should not be introduced.

---

36. Admin Frontend

Admin UI will be completely separate from the public marketing website.

Design:

> Clean SaaS dashboard.

No unnecessary animations.

Dashboard

Should show:

Total Teams
Total Participants
Verified Payments
Pending Payments
Registration Progress

---

37. Admin Management

Admins can manage:

- Teams
- Participants
- Payments

- Domains
- Schedule
- Prizes
- Sponsors
- Gallery
- FAQ
- Announcements

There will be:

One Admin role

No role hierarchy is required.

---

38. Gallery

The website should support approximately:

10–20 initial images

but the architecture should support considerably more.

Images can include:

- HackNEX branding
- NEXUS
- Campus
- Venue
- Sponsors
- Event photographs

Cloudinary will handle image storage, transformation and delivery.

Admins can:

- Upload
- Delete
- Replace
- Reorder
- Categorize

images.

---

39. Image Optimization

Frontend images should use:

- Responsive dimensions
- Lazy loading
- Appropriate compression
- Modern formats where available
- Cloudinary transformations

Large images should never be loaded at their original resolution when unnecessary.

---

40. Responsive Design

Supported devices:

Mobile
Tablet
Laptop
Desktop
Large Desktop

All sections must be responsive.

Particular attention should be given to:

- Hero typography
- Navbar
- Timeline
- Domain cards
- Sponsor logos
- Statistics
- Registration forms
- Authentication
- Admin dashboard

---

41. Animation System

Animation intensity:

Level 2–3

Approved animation types

- Fade
- Slide
- Scale
- Text reveal
- Scroll reveal
- Hover
- Magnetic buttons
- Parallax
- Particle effects
- Neural noise
- Subtle cursor effects

Avoid

- Excessive WebGL
- Continuous expensive animations
- Large video backgrounds throughout the site
- Animations that block interaction

---

42. Component Architecture

The frontend should use reusable components.

Example:

```
components/
│
├── layout/
│   ├── Navbar
│   ├── Footer
│   └── PageContainer
│
├── hero/
│   ├── Hero
│   ├── HeroBackground
│   └── Countdown
│
├── sections/
│   ├── About
│   ├── Domains
│   ├── Details
│   ├── Schedule
│   ├── Prizes
```

```
│   ├── Sponsors
│   ├── Venue
│   └── FAQ
│
├── ui/
│   ├── Button
│   ├── Card
│   ├── Modal
│   ├── Input
│   ├── Select
│   ├── Badge
│   └── Accordion
│
├── auth/
│
├── registration/
│
└── admin/
```

---

43. State Management

Frontend state will be separated into:

UI state

- Menu open/closed
- Modal
- Accordion
- Animation state

Authentication state

- Logged in
- Logged out
- Email verified
- Session

Registration state

- Team data
- Participant data
- Review state
- Payment state

Server state

- Teams
- Domains
- Schedule
- Sponsors
- FAQs
- Payment status

Server state should not unnecessarily be duplicated into local state.

---

44. API Integration

Frontend communicates with the backend through the defined API layer.

Architecture:

React Frontend
↓
API Client
↓
Backend API
↓
PostgreSQL

The frontend must not directly access the database.

---

45. Authentication Protection

Protected routes:

/register/*
/admin/*

The frontend should verify authentication state before rendering protected content.

However, frontend protection is **not a replacement for backend authorization**.

The backend remains the authoritative security layer.

---

46. Form Validation

Every form must provide:

- Required-field validation
- Format validation
- Email validation
- Phone validation
- LinkedIn URL validation
- Password requirements
- Duplicate participant detection
- Duplicate team detection

Validation errors must be shown near the relevant field.

---

47. Error States

Every API-driven component must support:

Loading
Success
Error
Empty
Retry

Example:

Unable to load schedule.

[Try Again]

The website must never leave users staring at an empty section when an API request fails.

---

48. Accessibility

The frontend should follow WCAG-oriented practices.

Requirements include:

- Semantic HTML
- Keyboard navigation
- Focus states
- Screen-reader-friendly labels
- Appropriate contrast

- Accessible forms
- Accessible dialogs
- Reduced-motion consideration
- Proper heading hierarchy
- Alternative text for meaningful images

---

49. SEO

The public website should include:

- Page title
- Meta description
- Open Graph metadata
- Social preview image
- Canonical URL
- Sitemap
- robots.txt
- Structured event data

The authentication and admin areas should not be indexed.

---

50. Analytics

Analytics will track important public-site behavior.

Potential metrics:

Visitors
Page views
Registration CTA clicks
Signup conversion
Registration conversion
Traffic sources

Analytics must not expose sensitive participant information.

---

51. Error Monitoring

Sentry will be integrated for:

- Frontend errors
- API errors
- Registration failures
- Payment-related frontend errors
- Unexpected application failures

Sensitive information must not be sent to monitoring services unnecessarily.

---

52. Performance Requirements

The website should prioritize performance because the expected audience is:

> 1,500+ participants

Requirements:

- Lazy loading
- Image optimization
- Cloudinary transformations
- Code splitting
- Component lazy loading where appropriate
- Optimized fonts
- Limited third-party scripts
- Efficient animations
- Minimal unnecessary JavaScript
- Caching where appropriate

---

53. Design Token System

Although exact colors will be finalized separately, the frontend should maintain centralized tokens for:

Primary
Secondary
Background
Surface
Surface Elevated
Text
Muted Text
Border

Accent
Success
Warning
Error

Spacing, radius, typography and shadows should also be tokenized.

This prevents individual components from developing inconsistent styling.

---

54. Typography System

A complete typography system will contain:

Display
H1
H2
H3
H4
Body Large
Body
Body Small
Caption
Button
Mono

The selected fonts should support:

- Strong futuristic headings
- Excellent readability
- Technical UI elements
- Mobile readability

---

55. Frontend Technology Direction

The frontend architecture should be built around:

React
TypeScript
Modern CSS / Tailwind CSS
Reusable UI components
21st.dev-inspired components
Framer Motion

Cloudinary
API client

The final implementation should remain modular and maintainable rather than embedding page-specific logic into giant components.

---

56. Security Considerations

Frontend must:

- Never contain API secrets
- Never contain database credentials
- Never trust client-side authorization
- Avoid exposing sensitive participant information
- Sanitize/render user-generated content safely
- Use secure authentication flows
- Handle expired sessions
- Avoid storing sensitive authentication information unnecessarily

Payment verification must always happen server-side.

---

57. Frontend Development Workflow

Implementation should follow this order:

1. Project setup
↓
2. Design tokens
↓
3. Global styles
↓
4. Navbar + Footer
↓
5. Loading screen
↓
6. Hero
↓
7. Public sections
↓
8. Responsive implementation
↓
9. Authentication UI

↓
10. Registration wizard
↓
11. Payment UI
↓
12. Admin dashboard
↓
13. API integration
↓
14. Cloudinary integration
↓
15. Analytics
↓
16. Sentry
↓
17. Testing
↓
18. Performance optimization
↓
19. SEO
↓
20. Production deployment

---

58. Final Public Website Flow

The complete user experience should be:

```
          VISITOR
             │
             ▼
      ┌─────────────────┐
      │ Loading Screen  │
      │ loadingscreen.mp4│
      └────────┬────────┘
             ▼
         LANDING PAGE
             │
  ┌──────────────┼──────────────┐
  ▼        ▼        ▼
Explore   Domains   Schedule
  │
  ▼
Prizes
  │
  ▼
```

```
   Sponsors
        |
        ▼
      FAQ
        |
        ▼
 REGISTER NOW
        |
        ▼
  LOGIN / SIGNUP
        |
        ▼
EMAIL VERIFICATION
        |
        ▼
REGISTRATION WIZARD
        |
        ▼
    REVIEW
        |
        ▼
   ₹600 PAYMENT
        |
        ▼
PAYMENT VERIFIED
        |
        ▼
REGISTRATION SUCCESS
        |
        ▼
CONFIRMATION EMAIL
        |
        ▼
FURTHER INFORMATION
      VIA EMAIL
```

---

59. Final Frontend Principle

The most important principle for HackNEX is:

> **Make the website feel premium and technically impressive without sacrificing usability, performance, or registration reliability.**

The visual experience can be experimental, but the actual registration flow must remain extremely clear.

A visitor should be able to understand **what HackNEX is, when it happens, where it happens, what they can win, and how to register** within a few seconds.

The landing page sells the event.

The registration flow completes the conversion.

The backend guarantees the correctness of the registration.

The email system handles everything after registration.